--TASK 1 USE INHERITANCE

The table/partitions/procedure/trigger were created and date inserted and distributed across partitions.

```sql
INSERT INTO sales_info(id, category, ischeck, EventDate)
SELECT id
     , ('{"A","B","C","D","E","F","J","H","I","J","K"}'::text[])[((RANDOM())*10)::INTEGER] category
     , ((1*(RANDOM())::INTEGER)<1) ischeck
     , (NOW() - (RANDOM() * 1600)::INTEGER * '1 day'::INTERVAL)::DATE EventDate
FROM generate_series(1,10000000) id;

SELECT count(*) FROM sales_info;

SELECT tableoid::regclass AS partition_name, count(*)
FROM sales_info
GROUP BY 1
ORDER BY 1;
```

ics 1 | Results 2 | Statistics 3 | Results 4

T tableoid::regclass AS partition_name, count | Enter a SQL expression to filter results (use Ctrl+Space)

partition_name	count
sales_info_2026	1,220,222
sales_info_2025	2,281,473
sales_info_2024	2,288,654
sales_info_2023	2,280,548
sales_info_2022	1,929,103

The 1000 rows were updated. Even though their eventdate is updated to 2024-06-15, physically they reside in sales_info_2022 that shows that inheritance-based partitioning does not rrelocate rows on UPDATE, since the trigger only fires on INSERT. This demonstrates a limitation of inheritance partitioning method.

```sql
UPDATE sales_info_2022
SET eventdate = '2024-06-15'
WHERE id IN (SELECT id FROM sales_info_2022 LIMIT 1000);

SELECT tableoid::regclass, eventdate, count(*)
FROM sales_info_2022
WHERE eventdate = '2024-06-15'
GROUP BY 1, 2;
```

cs 1 | Results 2 | Statistics 3 | Results 4 | sales_info_2022 6

T tableoid::regclass, eventdate, count(*) FROM | Enter a SQL expression to filter results (use Ctrl+Spac

tableoid	eventdate	count
sales_info_2022	2024-06-15	1,000

SELECT queries of the full table

On scan of the full table, the simple unpartitioned table has performance benefit over scanning all partitions

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM sales_info_simple;
```

```
QUERY PLAN
Seq Scan on sales_info_simple  (cost=0.00..151894.55 rows=9783955 width=17) (actual time=0.157..565.107 rows=10000000.00 loops=1)
  Buffers: shared hit=15768 read=38287
Planning Time: 0.954 ms
Execution Time: 906.433 ms
```

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM sales_info;
```

```
QUERY PLAN
Append  (cost=0.00..204075.88 rows=10000926 width=11) (actual time=0.076..1139.785 rows=10000000.00 loops=1)
  Buffers: shared read=54062
  ->  Seq Scan on sales_info sales_info_1  (cost=0.00..0.00 rows=1 width=17) (actual time=0.009..0.009 rows=0.00 loops=1)
  ->  Seq Scan on sales_info_2026 sales_info_2  (cost=0.00..18798.22 rows=1220222 width=11) (actual time=0.066..62.807 rows=1220222.00 loops=1)
        Buffers: shared read=6596
  ->  Seq Scan on sales_info_2025 sales_info_3  (cost=0.00..35147.73 rows=2281473 width=11) (actual time=0.199..116.073 rows=2281473.00 loops=1)
        Buffers: shared read=12333
  ->  Seq Scan on sales_info_2024 sales_info_4  (cost=0.00..35258.54 rows=2288654 width=11) (actual time=0.092..119.207 rows=2288654.00 loops=1)
        Buffers: shared read=12372
  ->  Seq Scan on sales_info_2023 sales_info_5  (cost=0.00..35133.48 rows=2280548 width=11) (actual time=0.136..119.591 rows=2280548.00 loops=1)
        Buffers: shared read=12328
  ->  Seq Scan on sales_info_2022 sales_info_6  (cost=0.00..29733.28 rows=1930028 width=11) (actual time=0.396..114.079 rows=1929103.00 loops=1)
        Buffers: shared read=10433
Planning:
  Buffers: shared read=2
Planning Time: 0.167 ms
Execution Time: 1495.537 ms
```

SELECT queries of the range

3 queries were performed: on simple table, partitioned before and after adding a CHECK constraint. Without CHECK constraints, inheritance-based partitioning has no advantage over a simple table. Once CHECK constraints were added to each partition, planner skipped not relevant partitions cutting execution time.

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM sales_info_simple
WHERE eventdate BETWEEN '2023-01-01' AND '2023-12-31';
```

s 1 | Results 2 | Statistics 3 | Results 4 | Statistics 7 | Results 8 | Results 9 | Results 10

(ANALYZE, BUFFERS) SELECT * FROM sales | Enter a SQL expression to filter results (use Ctrl+Space)

QUERY PLAN
Seq Scan on sales_info_simple (cost=0.00..204057.62 rows=2283735 width=11) (actual time=507.692..950.941 rows=2280548.00 loops=1)
Filter: ((eventdate >= '2023-01-01'::date) AND (eventdate <= '2023-12-31'::date))
Rows Removed by Filter: 7719452
Buffers: shared hit=14737 read=39318
Planning Time: 0.066 ms
Execution Time: 1037.906 ms

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM sales_info;
```

s 1 | Results 2 | Statistics 3 | Results 4 | sales_info_2022 6 | Statistics 7 | sales_info_simple 7 | Results 8

(ANALYZE, BUFFERS) SELECT * FROM sales | Enter a SQL expression to filter results (use Ctrl+Space)

QUERY PLAN
Append (cost=0.00..204075.88 rows=10000926 width=11) (actual time=0.076..1139.785 rows=10000000.00 loops=1)
Buffers: shared read=54062
-> Seq Scan on sales_info sales_info_1 (cost=0.00..0.00 rows=1 width=17) (actual time=0.009..0.009 rows=0.00 loops=1)
-> Seq Scan on sales_info_2026 sales_info_2 (cost=0.00..18798.22 rows=1220222 width=11) (actual time=0.066..62.807 rows=1220222.00 loops=1)
Buffers: shared read=6596
-> Seq Scan on sales_info_2025 sales_info_3 (cost=0.00..35147.73 rows=2281473 width=11) (actual time=0.199..116.073 rows=2281473.00 loops=1)
Buffers: shared read=12333
-> Seq Scan on sales_info_2024 sales_info_4 (cost=0.00..35258.54 rows=2288654 width=11) (actual time=0.092..119.207 rows=2288654.00 loops=1)
Buffers: shared read=12372
-> Seq Scan on sales_info_2023 sales_info_5 (cost=0.00..35133.48 rows=2280548 width=11) (actual time=0.136..119.591 rows=2280548.00 loops=1)
Buffers: shared read=12328
-> Seq Scan on sales_info_2022 sales_info_6 (cost=0.00..29733.28 rows=1930028 width=11) (actual time=0.396..114.079 rows=1929103.00 loops=1)
Buffers: shared read=10433
Planning:
Buffers: shared read=2
Planning Time: 0.167 ms
Execution Time: 1495.537 ms

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT count(*) FROM sales_info
WHERE eventdate BETWEEN '2023-01-01' AND '2023-12-31';
```

12 | Results 11 | Statistics 3 | Results 4 | Statistics 4

N (ANALYZE, BUFFERS) SELECT count(*) FRO | Enter a SQL expression to filter results (use Ctrl+Space)

```
QUERY PLAN
Finalize Aggregate  (cost=57202.65..57202.66 rows=1 width=8) (actual time=308.416..336.176 rows=1.00 loops=1)
  Buffers: shared hit=846 read=21915
  ->  Gather  (cost=57202.44..57202.65 rows=2 width=8) (actual time=304.077..336.169 rows=3.00 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=846 read=21915
        ->  Partial Aggregate  (cost=56202.44..56202.45 rows=1 width=8) (actual time=243.141..243.143 rows=1.00 loops=3)
              Buffers: shared hit=846 read=21915
              ->  Parallel Append  (cost=0.00..53827.33 rows=950045 width=0) (actual time=64.925..206.359 rows=760182.67 loops=3)
                    Buffers: shared hit=846 read=21915
                    ->  Parallel Seq Scan on sales_info_2023 sales_info_2  (cost=0.00..26581.43 rows=950043 width=0) (actual time=0.576..100.887 rows=760182.67 loops=3)
                          Filter: ((eventdate >= '2023-01-01'::date) AND (eventdate <= '2023-12-31'::date))
                          Buffers: shared hit=470 read=11858
                    ->  Parallel Seq Scan on sales_info_2022 sales_info_3  (cost=0.00..22495.67 rows=1 width=0) (actual time=96.514..96.514 rows=0.00 loops=2)
                          Filter: ((eventdate >= '2023-01-01'::date) AND (eventdate <= '2023-12-31'::date))
                          Rows Removed by Filter: 964552
                          Buffers: shared hit=376 read=10057
                    ->  Parallel Seq Scan on sales_info sales_info_1  (cost=0.00..0.00 rows=1 width=0) (actual time=0.003..0.003 rows=0.00 loops=1)
                          Filter: ((eventdate >= '2023-01-01'::date) AND (eventdate <= '2023-12-31'::date))
Planning:
  Buffers: shared hit=75
Planning Time: 3.173 ms
Execution Time: 336.233 ms
```

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM sales_info;
```

s 1 | Results 2 | Statistics 3 | Results 4 | sales_info_2022 6 | Statistics 7 | sales_info_simple 7 | Results 8

N (ANALYZE, BUFFERS) SELECT * FROM sales | Enter a SQL expression to filter results (use Ctrl+Space)

```
QUERY PLAN
Append  (cost=0.00..204075.88 rows=10000926 width=11) (actual time=0.076..1139.785 rows=10000000.00 loops=1)
  Buffers: shared read=54062
  ->  Seq Scan on sales_info sales_info_1  (cost=0.00..0.00 rows=1 width=17) (actual time=0.009..0.009 rows=0.00 loops=1)
  ->  Seq Scan on sales_info_2026 sales_info_2  (cost=0.00..18798.22 rows=1220222 width=11) (actual time=0.066..62.807 rows=1220222.00 loops=1)
        Buffers: shared read=6596
  ->  Seq Scan on sales_info_2025 sales_info_3  (cost=0.00..35147.73 rows=2281473 width=11) (actual time=0.199..116.073 rows=2281473.00 loops=1)
        Buffers: shared read=12333
  ->  Seq Scan on sales_info_2024 sales_info_4  (cost=0.00..35258.54 rows=2288654 width=11) (actual time=0.092..119.207 rows=2288654.00 loops=1)
        Buffers: shared read=12372
  ->  Seq Scan on sales_info_2023 sales_info_5  (cost=0.00..35133.48 rows=2280548 width=11) (actual time=0.136..119.591 rows=2280548.00 loops=1)
        Buffers: shared read=12328
  ->  Seq Scan on sales_info_2022 sales_info_6  (cost=0.00..29733.28 rows=1930028 width=11) (actual time=0.396..114.079 rows=1929103.00 loops=1)
        Buffers: shared read=10433
Planning:
  Buffers: shared read=2
Planning Time: 0.167 ms
Execution Time: 1495.537 ms
```

Select exact date.

Partitioned table scan showed better performance, not all rows were scanned

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM sales_info
WHERE eventdate = '2023-06-15';
```

12 | Results 11 | Results 4 | Results 4 | Results 5

N (ANALYZE, BUFFERS) SELECT count(*) FROM sales_info \ | Enter a SQL expression to filter results (use Ctrl+Space)

```
QUERY PLAN
Finalize Aggregate  (cost=57202.65..57202.66 rows=1 width=8) (actual time=445.050..490.672 rows=1.00 loops=1)
  Buffers: shared hit=1316 read=21445
  ->  Gather  (cost=57202.44..57202.65 rows=2 width=8) (actual time=444.883..490.665 rows=3.00 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=1316 read=21445
        ->  Partial Aggregate  (cost=56202.44..56202.45 rows=1 width=8) (actual time=329.836..329.838 rows=1.00 loops=3)
              Buffers: shared hit=1316 read=21445
              ->  Parallel Append  (cost=0.00..53827.33 rows=950045 width=0) (actual time=79.100..280.457 rows=760182.67 loops=3)
                    Buffers: shared hit=1316 read=21445
                    ->  Parallel Seq Scan on sales_info_2023 sales_info_2  (cost=0.00..26581.43 rows=950043 width=0) (actual time=0.960..150.418 rows=760182.67 loops=3)
                          Filter: ((eventdate >= '2023-01-01'::date) AND (eventdate <= '2023-12-31'::date))
                          Buffers: shared hit=752 read=11576
                    ->  Parallel Seq Scan on sales_info_2022 sales_info_3  (cost=0.00..22495.67 rows=1 width=0) (actual time=117.196..117.196 rows=0.00 loops=2)
                          Filter: ((eventdate >= '2023-01-01'::date) AND (eventdate <= '2023-12-31'::date))
                          Rows Removed by Filter: 964552
                          Buffers: shared hit=564 read=9869
                    ->  Parallel Seq Scan on sales_info sales_info_1  (cost=0.00..0.00 rows=1 width=0) (actual time=0.002..0.003 rows=0.00 loops=1)
                          Filter: ((eventdate >= '2023-01-01'::date) AND (eventdate <= '2023-12-31'::date))
Planning:
  Buffers: shared hit=184 read=2
Planning Time: 5.221 ms
Execution Time: 490.749 ms
```

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT count(*) FROM sales_info_simple
WHERE eventdate BETWEEN '2023-01-01' AND '2023-12-31';
```

12 | Results 11 | Results 4 | Results 4 | Results 5

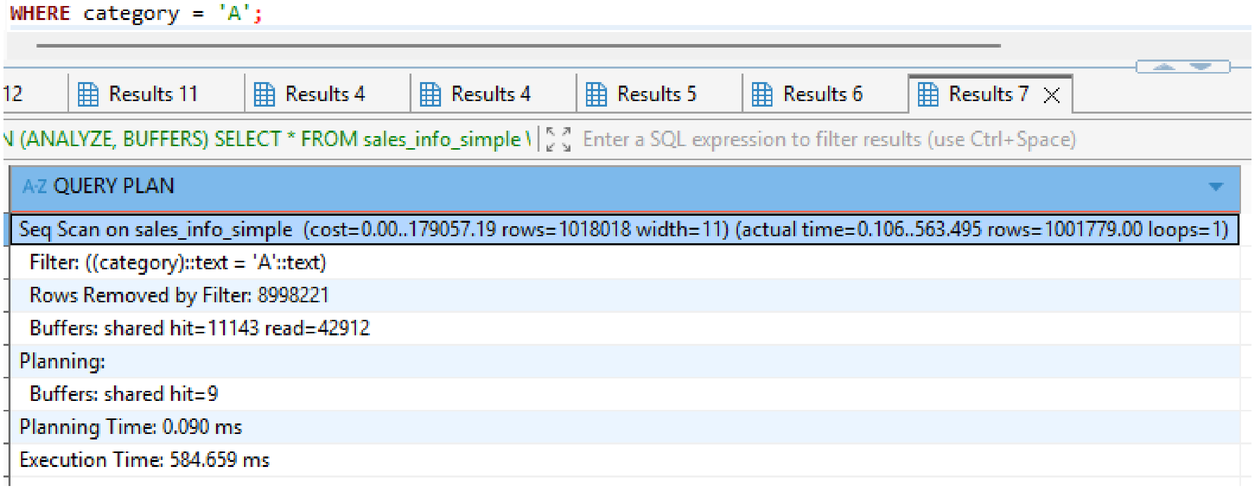
IN (ANALYZE, BUFFERS) SELECT count(*) FRO | Enter a SQL expression to filter results (use Ctrl+Space)

```
QUERY PLAN
Finalize Aggregate  (cost=119935.20..119935.21 rows=1 width=8) (actual time=610.228..637.096 rows=1.00 loops=1)
  Buffers: shared hit=12218 read=41837
  ->  Gather  (cost=119934.98..119935.19 rows=2 width=8) (actual time=609.930..637.088 rows=3.00 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=12218 read=41837
        ->  Partial Aggregate  (cost=118934.98..118934.99 rows=1 width=8) (actual time=567.427..567.427 rows=1.00 loops=3)
              Buffers: shared hit=12218 read=41837
              ->  Parallel Seq Scan on sales_info_simple  (cost=0.00..116556.09 rows=951556 width=0) (actual time=308.295..523.112 rows=760182.67 loops=3)
                    Filter: ((eventdate >= '2023-01-01'::date) AND (eventdate <= '2023-12-31'::date))
                    Rows Removed by Filter: 2573151
                    Buffers: shared hit=12218 read=41837
Planning:
  Buffers: shared hit=5
Planning Time: 1.119 ms
Execution Time: 638.062 ms
```

Select exact category/list of categories

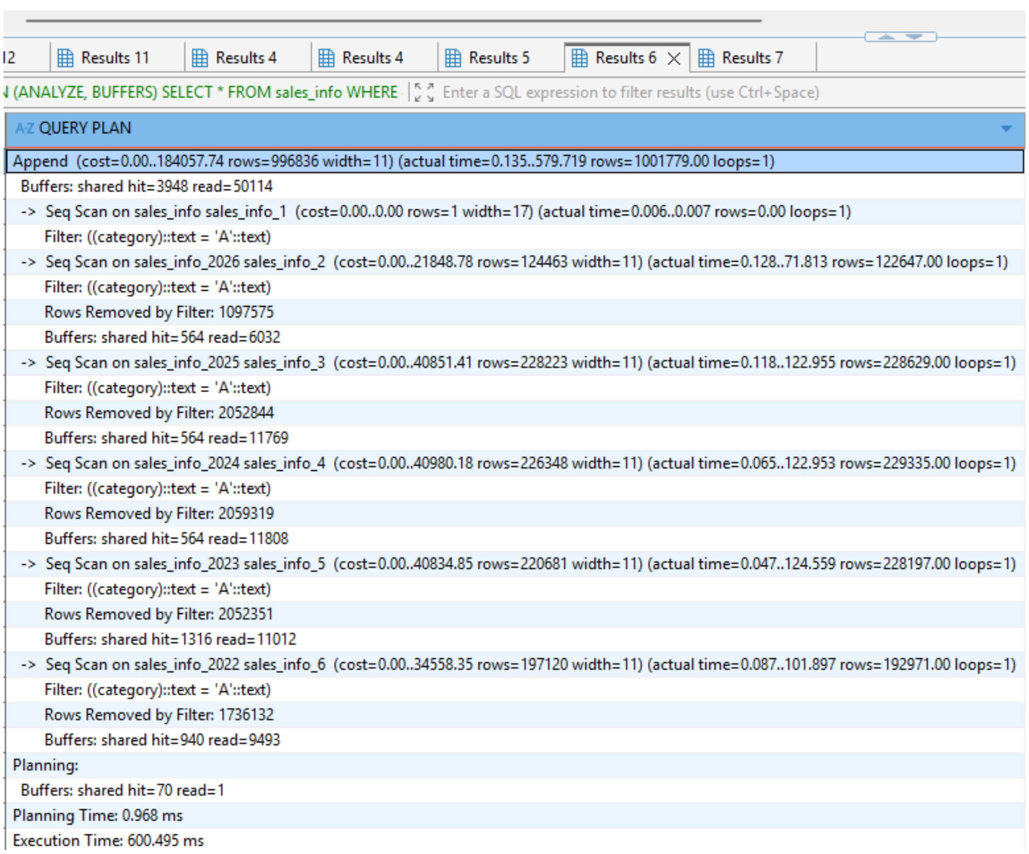
As the partitioning was made on date not category, the scan on partition tables has no benefit over simple full table scan. The same behavior is observed with a list of categories

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM sales_info_simple
WHERE category = 'A';
```




```
Seq Scan on sales_info_simple  (cost=0.00..179057.19 rows=1018018 width=11) (actual time=0.106..563.495 rows=1001779.00 loops=1)
  Filter: ((category)::text = 'A'::text)
  Rows Removed by Filter: 8998221
  Buffers: shared hit=11143 read=42912
Planning:
  Buffers: shared hit=9
Planning Time: 0.090 ms
Execution Time: 584.659 ms
```

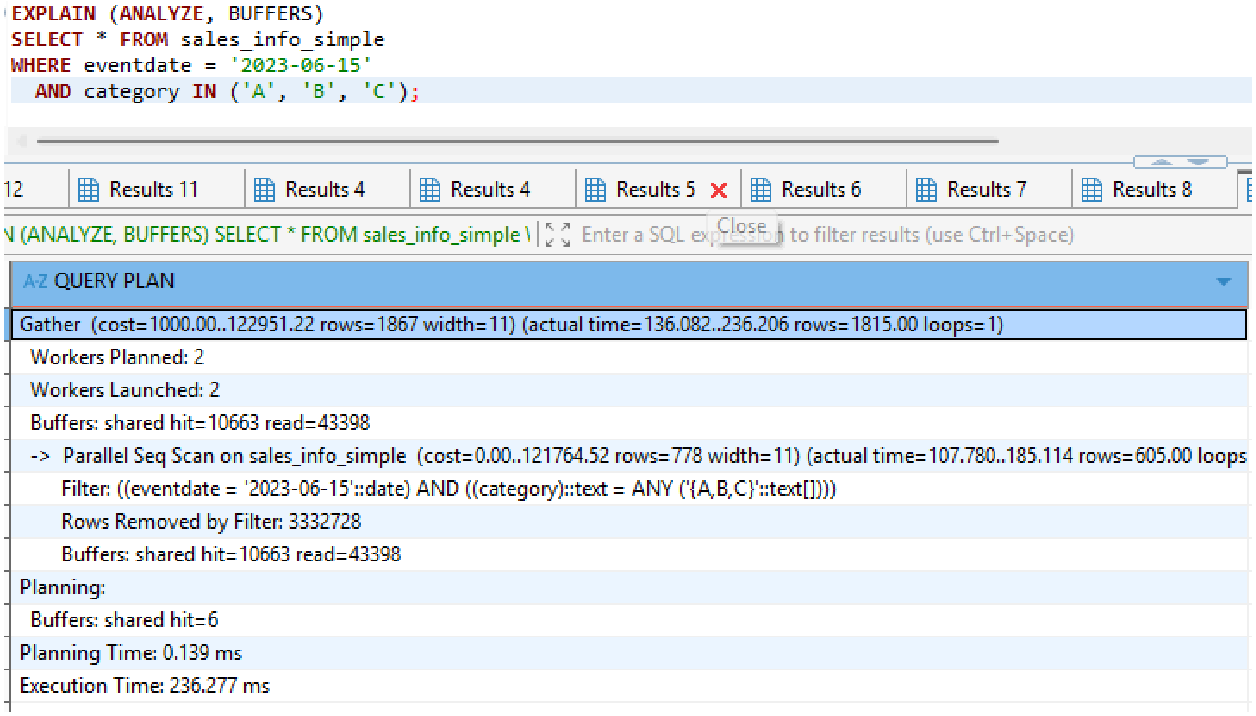
```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM sales_info
WHERE category = 'A';
```




```
Append  (cost=0.00..184057.74 rows=996836 width=11) (actual time=0.135..579.719 rows=1001779.00 loops=1)
  Buffers: shared hit=3948 read=50114
  ->  Seq Scan on sales_info sales_info_1  (cost=0.00..0.00 rows=1 width=17) (actual time=0.006..0.007 rows=0.00 loops=1)
        Filter: ((category)::text = 'A'::text)
  ->  Seq Scan on sales_info_2026 sales_info_2  (cost=0.00..21848.78 rows=124463 width=11) (actual time=0.128..71.813 rows=122647.00 loops=1)
        Filter: ((category)::text = 'A'::text)
        Rows Removed by Filter: 1097575
        Buffers: shared hit=564 read=6032
  ->  Seq Scan on sales_info_2025 sales_info_3  (cost=0.00..40851.41 rows=228223 width=11) (actual time=0.118..122.955 rows=228629.00 loops=1)
        Filter: ((category)::text = 'A'::text)
        Rows Removed by Filter: 2052844
        Buffers: shared hit=564 read=11769
  ->  Seq Scan on sales_info_2024 sales_info_4  (cost=0.00..40980.18 rows=226348 width=11) (actual time=0.065..122.953 rows=229335.00 loops=1)
        Filter: ((category)::text = 'A'::text)
        Rows Removed by Filter: 2059319
        Buffers: shared hit=564 read=11808
  ->  Seq Scan on sales_info_2023 sales_info_5  (cost=0.00..40834.85 rows=220681 width=11) (actual time=0.047..124.559 rows=228197.00 loops=1)
        Filter: ((category)::text = 'A'::text)
        Rows Removed by Filter: 2052351
        Buffers: shared hit=1316 read=11012
  ->  Seq Scan on sales_info_2022 sales_info_6  (cost=0.00..34558.35 rows=197120 width=11) (actual time=0.087..101.897 rows=192971.00 loops=1)
        Filter: ((category)::text = 'A'::text)
        Rows Removed by Filter: 1736132
        Buffers: shared hit=940 read=9493
Planning:
  Buffers: shared hit=70 read=1
Planning Time: 0.968 ms
Execution Time: 600.495 ms
```

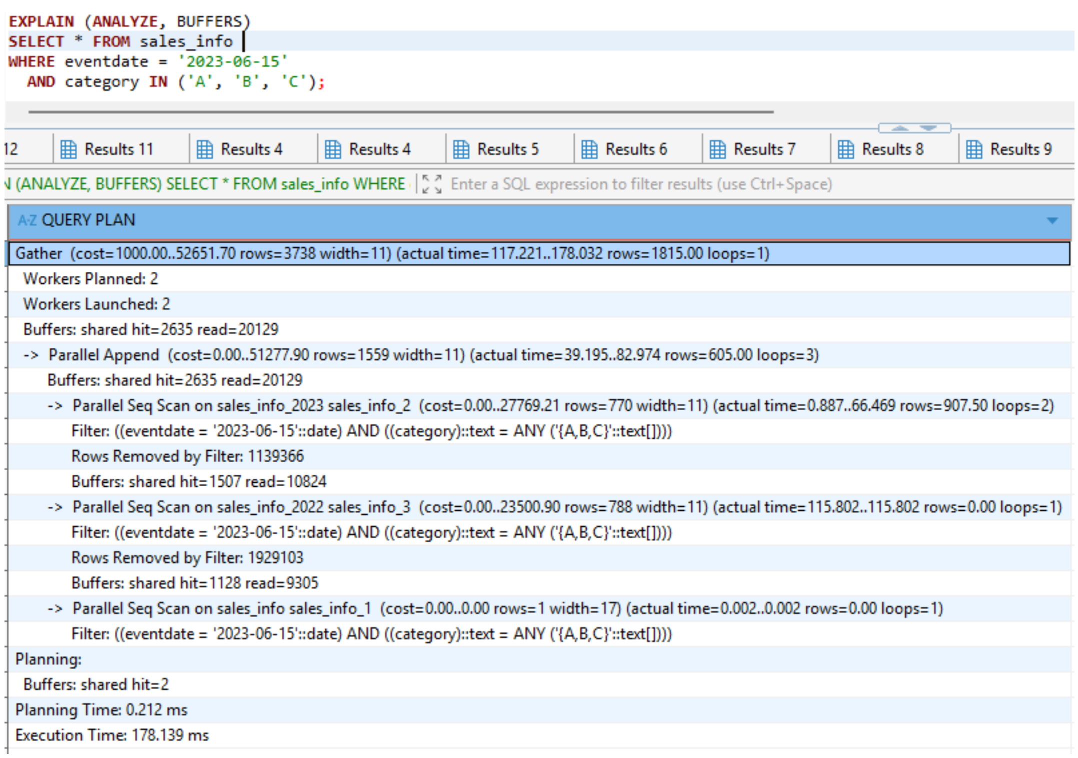
Select a list of categories in exact date

Even though filtering was on a non-partitioned column (category), partial filter by the partition key (eventdate), showed performance benefit.



```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM sales_info_simple
WHERE eventdate = '2023-06-15'
  AND category IN ('A', 'B', 'C');
```

```
QUERY PLAN
Gather  (cost=1000.00..122951.22 rows=1867 width=11) (actual time=136.082..236.206 rows=1815.00 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  Buffers: shared hit=10663 read=43398
  ->  Parallel Seq Scan on sales_info_simple  (cost=0.00..121764.52 rows=778 width=11) (actual time=107.780..185.114 rows=605.00 loops
        Filter: ((eventdate = '2023-06-15'::date) AND ((category)::text = ANY ('{A,B,C}'::text[])))
        Rows Removed by Filter: 3332728
        Buffers: shared hit=10663 read=43398
Planning:
  Buffers: shared hit=6
Planning Time: 0.139 ms
Execution Time: 236.277 ms
```



```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM sales_info
WHERE eventdate = '2023-06-15'
  AND category IN ('A', 'B', 'C');
```

```
QUERY PLAN
Gather  (cost=1000.00..52651.70 rows=3738 width=11) (actual time=117.221..178.032 rows=1815.00 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  Buffers: shared hit=2635 read=20129
  ->  Parallel Append  (cost=0.00..51277.90 rows=1559 width=11) (actual time=39.195..82.974 rows=605.00 loops=3)
        Buffers: shared hit=2635 read=20129
        ->  Parallel Seq Scan on sales_info_2023 sales_info_2  (cost=0.00..27769.21 rows=770 width=11) (actual time=0.887..66.469 rows=907.50 loops=2)
              Filter: ((eventdate = '2023-06-15'::date) AND ((category)::text = ANY ('{A,B,C}'::text[])))
              Rows Removed by Filter: 1139366
              Buffers: shared hit=1507 read=10824
        ->  Parallel Seq Scan on sales_info_2022 sales_info_3  (cost=0.00..23500.90 rows=788 width=11) (actual time=115.802..115.802 rows=0.00 loops=1)
              Filter: ((eventdate = '2023-06-15'::date) AND ((category)::text = ANY ('{A,B,C}'::text[])))
              Rows Removed by Filter: 1929103
              Buffers: shared hit=1128 read=9305
        ->  Parallel Seq Scan on sales_info sales_info_1  (cost=0.00..0.00 rows=1 width=17) (actual time=0.002..0.002 rows=0.00 loops=1)
              Filter: ((eventdate = '2023-06-15'::date) AND ((category)::text = ANY ('{A,B,C}'::text[])))
Planning:
  Buffers: shared hit=2
Planning Time: 0.212 ms
Execution Time: 178.139 ms
```

Count of all rows

A flat table is more efficient because it avoids the extra work of coordinating queries across multiple partition files.

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT count(*) FROM sales_info_simple;
```

12 | Results 11 | Results 4 | Results 4 | Results 5 | Results 6 | Results 7 | Results 8 | Results 9

N (ANALYZE, BUFFERS) SELECT count(*) FROM sales_info_s

```
QUERY PLAN
Finalize Aggregate  (cost=107139.46..107139.47 rows=1 width=8) (actual time=265.640..308.238 rows=1.00 loops=1)
  Buffers: shared hit=10614 read=43441
  ->  Gather  (cost=107139.25..107139.46 rows=2 width=8) (actual time=265.524..308.231 rows=3.00 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=10614 read=43441
        ->  Partial Aggregate  (cost=106139.25..106139.26 rows=1 width=8) (actual time=238.771..238.776 rows=1.00 loops=3)
              Buffers: shared hit=10614 read=43441
              ->  Parallel Seq Scan on sales_info_simple  (cost=0.00..95722.40 rows=4166740 width=0) (actual time=0.451..124.433 rows=3333333.33 loops=3)
                    Buffers: shared hit=10614 read=43441
Planning Time: 0.091 ms
```

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT count(*) FROM sales_info;
```

12 | Results 11 | Results 4 | Results 4 | Results 5 | Results 6 | Results 7 | Results 8 | Results 9 | Results

N (ANALYZE, BUFFERS) SELECT count(*) FROM sales_info | Enter a SQL expression to filter results (use Ctrl+Space)

```
QUERY PLAN
Finalize Aggregate  (cost=127985.63..127985.64 rows=1 width=8) (actual time=478.382..501.071 rows=1.00 loops=1)
  Buffers: shared hit=4335 read=49727
  ->  Gather  (cost=127985.42..127985.63 rows=2 width=8) (actual time=478.190..501.066 rows=3.00 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=4335 read=49727
        ->  Partial Aggregate  (cost=126985.42..126985.43 rows=1 width=8) (actual time=415.479..415.480 rows=1.00 loops=3)
              Buffers: shared hit=4335 read=49727
              ->  Parallel Append  (cost=0.00..116567.79 rows=4167053 width=0) (actual time=0.443..296.358 rows=3333333.33 loops=3)
                    Buffers: shared hit=4335 read=49727
                    ->  Parallel Seq Scan on sales_info_2024 sales_info_4  (cost=0.00..21908.06 rows=953606 width=0) (actual time=0.546..122.906 rows=2288654.00 loops=1)
                          Buffers: shared hit=564 read=11808
                    ->  Parallel Seq Scan on sales_info_2025 sales_info_3  (cost=0.00..21839.14 rows=950614 width=0) (actual time=0.658..108.885 rows=2281473.00 loops=1)
                          Buffers: shared hit=298 read=12035
                    ->  Parallel Seq Scan on sales_info_2023 sales_info_5  (cost=0.00..21830.28 rows=950228 width=0) (actual time=0.450..37.211 rows=760182.67 loops=3)
                          Buffers: shared hit=1692 read=10636
                    ->  Parallel Seq Scan on sales_info_2022 sales_info_6  (cost=0.00..18474.78 rows=804178 width=0) (actual time=0.003..91.886 rows=1929103.00 loops=1)
                          Buffers: shared hit=1222 read=9211
                    ->  Parallel Seq Scan on sales_info_2026 sales_info_2  (cost=0.00..11680.26 rows=508426 width=0) (actual time=0.115..67.046 rows=1220222.00 loops=1)
                          Buffers: shared hit=559 read=6037
                    ->  Parallel Seq Scan on sales_info sales_info_1  (cost=0.00..0.00 rows=1 width=0) (actual time=0.004..0.004 rows=0.00 loops=1)
Planning:
  Buffers: shared hit=2
Planning Time: 0.158 ms
Execution Time: 501.113 ms
```

Count of rows with range of dates

Partitioning provides performance benefit because the scan limits strictly to the partitions containing the targeted date range.

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT count(*) FROM sales_info_simple;
```

```
QUERY PLAN
Finalize Aggregate  (cost=119935.20..119935.21 rows=1 width=8) (actual time=304.443..310.192 rows=1.00 loops=1)
  Buffers: shared hit=10896 read=43159
  ->  Gather  (cost=119934.98..119935.19 rows=2 width=8) (actual time=304.330..310.187 rows=3.00 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=10896 read=43159
        ->  Partial Aggregate  (cost=118934.98..118934.99 rows=1 width=8) (actual time=246.389..246.390 rows=1.00 loops=3)
              Buffers: shared hit=10896 read=43159
              ->  Parallel Seq Scan on sales_info_simple  (cost=0.00..116556.09 rows=951556 width=0) (actual time=121.390..221.349 rows=760182.67 loops=3)
                    Filter: ((eventdate >= '2023-01-01'::date) AND (eventdate <= '2023-12-31'::date))
                    Rows Removed by Filter: 2573151
                    Buffers: shared hit=10896 read=43159
Planning Time: 0.058 ms
Execution Time: 310.216 ms
```

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT count(*) FROM sales_info
WHERE eventdate BETWEEN '2023-01-01' AND '2023-12-31';
```

```
QUERY PLAN
Finalize Aggregate  (cost=57202.65..57202.66 rows=1 width=8) (actual time=184.696..208.883 rows=1.00 loops=1)
  Buffers: shared hit=3478 read=19283
  ->  Gather  (cost=57202.44..57202.65 rows=2 width=8) (actual time=184.591..208.878 rows=3.00 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=3478 read=19283
        ->  Partial Aggregate  (cost=56202.44..56202.45 rows=1 width=8) (actual time=160.309..160.310 rows=1.00 loops=3)
              Buffers: shared hit=3478 read=19283
              ->  Parallel Append  (cost=0.00..53827.33 rows=950045 width=0) (actual time=41.454..135.270 rows=760182.67 loops=3)
                    Buffers: shared hit=3478 read=19283
                    ->  Parallel Seq Scan on sales_info_2023 sales_info_2  (cost=0.00..26581.43 rows=950043 width=0) (actual time=0.461..65.807 rows=760182.67 loops=3)
                          Filter: ((eventdate >= '2023-01-01'::date) AND (eventdate <= '2023-12-31'::date))
                          Buffers: shared hit=2068 read=10260
                    ->  Parallel Seq Scan on sales_info_2022 sales_info_3  (cost=0.00..22495.67 rows=1 width=0) (actual time=61.485..61.485 rows=0.00 loops=2)
                          Filter: ((eventdate >= '2023-01-01'::date) AND (eventdate <= '2023-12-31'::date))
                          Rows Removed by Filter: 964552
                          Buffers: shared hit=1410 read=9023
                    ->  Parallel Seq Scan on sales_info sales_info_1  (cost=0.00..0.00 rows=1 width=0) (actual time=0.002..0.002 rows=0.00 loops=1)
                          Filter: ((eventdate >= '2023-01-01'::date) AND (eventdate <= '2023-12-31'::date))
Planning:
  Buffers: shared hit=2
Planning Time: 0.168 ms
Execution Time: 208.915 ms
```

Finally, one partition is deleted and another is created

tableoid	count
sales_info_2026	1,220,222
sales_info_2025	2,281,473
sales_info_2024	2,288,654
sales_info_2023	2,280,548
sales_info_3000	1

--TASK 2 USE DECLARATIVE PARTITIONING

The table for declarative partitions is created, data are inserted, new partition is created (update query)
Results of the SELECT queries:

• **Select all**

The single table query scanned one single physical relation using a sequential scan.
The partitioned table query scans across all 12 nested partitions that is slower.

EXPLAIN ANALYZE SELECT * FROM labs.sales_info_simple | Enter a SQL expression to filter results (use Ctrl+Space)

	QUERY PLAN
1	Seq Scan on sales_info_simple (cost=0.00..154056.75 rows=10000175 width=11) (actual time=0.107..300.117 rows=10000000.00 loops=1)
2	Buffers: shared hit=94 read=53961
3	Planning Time: 0.055 ms
4	Execution Time: 506.543 ms

EXPLAIN ANALYZE SELECT * FROM labs.sales_info_dp

	QUERY PLAN
1	Append (cost=0.00..204105.51 rows=10001234 width=11) (actual time=0.153..660.243 rows=10000000.00 loops=1)
2	Buffers: shared hit=13092 read=40995 dirtied=8
3	-> Seq Scan on sales_info_dp_2021_list1 sales_info_dp_1 (cost=0.00..18033.87 rows=1170587 width=11) (actual time=0.152..36.656 rows=1169974.00 loops=1)
4	Buffers: shared hit=1 read=6327 dirtied=4
5	-> Seq Scan on sales_info_dp_2021_list2 sales_info_dp_2 (cost=0.00..20086.37 rows=1303837 width=11) (actual time=0.091..44.119 rows=1303216.00 loops=1)
6	Buffers: shared hit=658 read=6390 dirtied=4
7	-> Seq Scan on sales_info_dp_2021_default sales_info_dp_3 (cost=0.00..2010.04 rows=130404 width=11) (actual time=0.101..5.314 rows=130404.00 loops=1)
8	Buffers: shared read=706
9	-> Seq Scan on sales_info_dp_2022_list1 sales_info_dp_4 (cost=0.00..18112.91 rows=1175491 width=11) (actual time=0.111..36.054 rows=1175491.00 loops=1)
10	Buffers: shared read=6358
11	-> Seq Scan on sales_info_dp_2022_list2 sales_info_dp_5 (cost=0.00..20073.50 rows=1302750 width=11) (actual time=0.098..39.356 rows=1302750.00 loops=1)
12	Buffers: shared hit=1 read=7045
13	-> Seq Scan on sales_info_dp_2022_default sales_info_dp_6 (cost=0.00..1997.10 rows=129610 width=11) (actual time=0.042..4.121 rows=129610.00 loops=1)
14	Buffers: shared read=701
15	-> Seq Scan on sales_info_dp_2023_list1 sales_info_dp_7 (cost=0.00..18076.23 rows=1173123 width=11) (actual time=0.037..34.759 rows=1173123.00 loops=1)
16	Buffers: shared read=6345
17	-> Seq Scan on sales_info_dp_2023_list2 sales_info_dp_8 (cost=0.00..20102.14 rows=1304614 width=11) (actual time=0.116..41.409 rows=1304614.00 loops=1)
18	Buffers: shared hit=4270 read=2786
19	-> Seq Scan on sales_info_dp_2023_default sales_info_dp_9 (cost=0.00..2001.69 rows=129869 width=11) (actual time=0.037..4.267 rows=129869.00 loops=1)
20	Buffers: shared read=703
21	-> Seq Scan on sales_info_dp_2024_list1 sales_info_dp_10 (cost=0.00..15098.25 rows=979825 width=11) (actual time=0.093..29.023 rows=979825.00 loops=1)
22	Buffers: shared hit=2836 read=2464
23	-> Seq Scan on sales_info_dp_2024_list2 sales_info_dp_11 (cost=0.00..16835.31 rows=1092631 width=11) (actual time=0.013..34.288 rows=1092631.00 loops=1)
24	Buffers: shared hit=4739 read=1170
25	-> Seq Scan on sales_info_dp_2024_default sales_info_dp_12 (cost=0.00..1671.93 rows=108493 width=11) (actual time=0.014..3.132 rows=108493.00 loops=1)
26	Buffers: shared hit=587
27	Planning Time: 0.173 ms
28	Execution Time: 863.770 ms

- **Select with range of dates**

The flat table did a Seq Scan on all 10 million rows.The partitioned table scanned only 3 partitions**.**

```sql
EXPLAIN ANALYZE
SELECT *
FROM labs.sales_info_simple
WHERE eventdate BETWEEN '2022-01-01' AND '2022-06-01';
```

1 | Results 6 | Results 6

N ANALYZE SELECT * FROM labs.sales | Enter a SQL expression to filter results (use Ctrl+Space)

```
QUERY PLAN
Seq Scan on sales_info_simple  (cost=0.00..154056.75 rows=10000175 width=11) (actual time=0.103..304.460 rows=10000000.00 loops=1)
  Buffers: shared hit=470 read=53585
Planning Time: 0.037 ms
Execution Time: 510.560 ms
```

```sql
EXPLAIN ANALYZE
SELECT *
FROM labs.sales_info_dp
WHERE eventdate = '2023-04-12';
```

1 | Results 6 | Results 6

N ANALYZE SELECT * FROM labs.sales | Enter a SQL expression to filter results (use Ctrl+Space)

```
QUERY PLAN
Append  (cost=0.00..58600.21 rows=1075489 width=11) (actual time=0.014..203.272 rows=1084994.00 loops=1)
  Buffers: shared hit=1085 read=13020
  ->  Seq Scan on sales_info_dp_2022_list1 sales_info_dp_1  (cost=0.00..23990.36 rows=491473 width=11) (actual time=0.013..75.914 rows=488781.00 loops=1)
        Filter: ((eventdate >= '2022-01-01'::date) AND (eventdate <= '2022-06-01'::date))
        Rows Removed by Filter: 686710
        Buffers: shared hit=195 read=6163
  ->  Seq Scan on sales_info_dp_2022_list2 sales_info_dp_2  (cost=0.00..26587.25 rows=530641 width=11) (actual time=0.097..80.817 rows=542276.00 loops=1)
        Filter: ((eventdate >= '2022-01-01'::date) AND (eventdate <= '2022-06-01'::date))
        Rows Removed by Filter: 760474
        Buffers: shared hit=189 read=6857
  ->  Seq Scan on sales_info_dp_2022_default sales_info_dp_3  (cost=0.00..2645.15 rows=53375 width=11) (actual time=0.016..8.152 rows=53937.00 loops=1)
        Filter: ((eventdate >= '2022-01-01'::date) AND (eventdate <= '2022-06-01'::date))
        Rows Removed by Filter: 75673
        Buffers: shared hit=701
Planning Time: 0.117 ms
Execution Time: 226.042 ms
```

- **Select exact date**

The flat table sequentially scanned the entire dataset.
The partitioned table scanned only the 3 subpartitions for the year 2023.

```sql
EXPLAIN ANALYZE
SELECT *
FROM labs.sales_info_simple
WHERE eventdate = '2023-04-12';
```

1 | Results 2 | Results 6 | Results 4 | Results 5

N ANALYZE SELECT * FROM labs.sales_info_simple WHERE | Enter a SQL expression to filter results (use Ctrl+Space)

```
QUERY PLAN
Gather  (cost=1000.00..107761.04 rows=6218 width=11) (actual time=177.653..291.227 rows=6113.00 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  Buffers: shared hit=846 read=53209
  ->  Parallel Seq Scan on sales_info_simple  (cost=0.00..106139.24 rows=2591 width=11) (actual time=116.304..200.471 rows=2037.67 loops=3)
        Filter: (eventdate = '2023-04-12'::date)
        Rows Removed by Filter: 3331296
        Buffers: shared hit=846 read=53209
Planning Time: 0.087 ms
Execution Time: 291.460 ms
```

```sql
EXPLAIN ANALYZE
SELECT *
FROM labs.sales_info_dp
WHERE eventdate = '2023-04-12';
```

Results 2 | Results 6 | Results 4

I ANALYZE SELECT * FROM labs.sales_info_dp WHERE eve | Enter a SQL expression to filter results (use Ctrl+Space)

```
QUERY PLAN
Gather  (cost=1000.00..29689.00 rows=7104 width=11) (actual time=0.389..110.608 rows=7213.00 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  Buffers: shared hit=4593 read=9511
  ->  Parallel Append  (cost=0.00..27978.60 rows=2960 width=11) (actual time=0.357..61.901 rows=2404.33 loops=3)
        Buffers: shared hit=4593 read=9511
        ->  Parallel Seq Scan on sales_info_dp_2023_list2 sales_info_dp_2  (cost=0.00..13850.86 rows=1477 width=11) (actual time=0.651..29.117 rov
              Filter: (eventdate = '2023-04-12'::date)
              Rows Removed by Filter: 433662
              Buffers: shared hit=3702 read=3354
        ->  Parallel Seq Scan on sales_info_dp_2023_list1 sales_info_dp_1  (cost=0.00..12455.02 rows=1336 width=11) (actual time=0.365..45.087 rov
              Filter: (eventdate = '2023-04-12'::date)
              Rows Removed by Filter: 584954
              Buffers: shared hit=188 read=6157
        ->  Parallel Seq Scan on sales_info_dp_2023_default sales_info_dp_3  (cost=0.00..1657.92 rows=208 width=11) (actual time=0.060..7.614 row
              Filter: (eventdate = '2023-04-12'::date)
              Rows Removed by Filter: 129501
              Buffers: shared hit=703
Planning Time: 0.117 ms
Execution Time: 110.926 ms
```

- **Select exact category**

The flat table sequentially scanned everything.
The partitioned table bypassed the default and List 2 subpartitions, scanning only the List 1 subpartitions across all 4 years.
The same for the list of categories.

```sql
EXPLAIN ANALYZE
SELECT *
FROM labs.sales_info_simple
WHERE category = 'A';
```

1 | Results 2 | Results 6 | Results 4 | Results 5 | Results 6

N ANALYZE SELECT * FROM labs.sales_info_simple WHERE | Enter a SQL expression to filter results (use Ctrl+Space)

```
QUERY PLAN
Seq Scan on sales_info_simple  (cost=0.00..179057.19 rows=1018018 width=11) (actual time=0.108..527.220 rows=1001779.00 loops=1)
  Filter: ((category)::text = 'A'::text)
  Rows Removed by Filter: 8998221
  Buffers: shared hit=1128 read=52927
Planning Time: 0.043 ms
Execution Time: 548.434 ms
```

```sql
EXPLAIN ANALYZE
SELECT *
FROM labs.sales_info_dp
WHERE category = 'A';
```

1 | Results 2 | Results 6 | Results 4 | Results 5

N ANALYZE SELECT * FROM labs.sales_info_dp WHERE cat | Enter a SQL expression to filter results (use Ctrl+Space)

```
QUERY PLAN
Append  (cost=0.00..83095.18 rows=505270 width=11) (actual time=0.101..266.423 rows=499714.00 loops=1)
  Buffers: shared hit=3814 read=20517
  -> Seq Scan on sales_info_dp_2021_list1 sales_info_dp_1  (cost=0.00..20960.34 rows=130169 width=11) (actual time=0.100..64.524 rows=12958
      Filter: ((category)::text = 'A'::text)
      Rows Removed by Filter: 1040389
      Buffers: shared hit=94 read=6234
  -> Seq Scan on sales_info_dp_2022_list1 sales_info_dp_2  (cost=0.00..21051.64 rows=131028 width=11) (actual time=0.016..63.752 rows=13038
      Filter: ((category)::text = 'A'::text)
      Rows Removed by Filter: 1045109
      Buffers: shared hit=383 read=5975
  -> Seq Scan on sales_info_dp_2023_list1 sales_info_dp_3  (cost=0.00..21009.04 rows=132798 width=11) (actual time=0.054..65.757 rows=13053
      Filter: ((category)::text = 'A'::text)
      Rows Removed by Filter: 1042590
      Buffers: shared hit=376 read=5969
  -> Seq Scan on sales_info_dp_2024_list1 sales_info_dp_4  (cost=0.00..17547.81 rows=111275 width=11) (actual time=0.101..54.464 rows=10921
      Filter: ((category)::text = 'A'::text)
      Rows Removed by Filter: 870611
      Buffers: shared hit=2961 read=2339
Planning Time: 0.226 ms
Execution Time: 279.144 ms
```

Select a list of categories in exact date

The flat table did a full sequential scan.The partitioned table scanned exactly 1 single leaf partition: sales_info_dp_2023_list1

```sql
EXPLAIN ANALYZE
SELECT *
FROM labs.sales_info_simple
WHERE eventdate = '2023-04-12'
  AND category IN ('A', 'B');
```

```
QUERY PLAN
Gather  (cost=1000.00..117681.09 rows=1250 width=11) (actual time=252.415..448.731 rows=1234.00 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  Buffers: shared hit=1322 read=52739
  ->  Parallel Seq Scan on sales_info_simple  (cost=0.00..116556.09 rows=521 width=11) (actual time=194.838..365.251 rows=411.33 loops=3)
        Filter: (((category)::text = ANY ('{A,B}'::text[])) AND (eventdate = '2023-04-12'::date))
        Rows Removed by Filter: 3332922
        Buffers: shared hit=1322 read=52739
Planning Time: 0.102 ms
```

```sql
EXPLAIN ANALYZE
SELECT *
FROM labs.sales_info_dp
WHERE eventdate = '2023-04-12'
  AND category IN ('A', 'B');
```

```
QUERY PLAN
Gather  (cost=1000.00..14783.22 rows=1062 width=11) (actual time=1.028..98.353 rows=1080.00 loops=1)
  Workers Planned: 2
  Workers Launched: 2
  Buffers: shared hit=570 read=5781
  ->  Parallel Seq Scan on sales_info_dp_2023_list1 sales_info_dp  (cost=0.00..13677.02 rows=442 width=11) (actual time=0.665..44.543 rows=360
        Filter: (((category)::text = ANY ('{A,B}'::text[])) AND (eventdate = '2023-04-12'::date))
        Rows Removed by Filter: 390681
        Buffers: shared hit=570 read=5781
Planning Time: 0.184 ms
Execution Time: 98.438 ms
```

• **Count of all rows**

Flat table performed a highly optimized Parallel Append and Aggregate scan.

Partitioned Table: Scanned all 12 nested leaf partitions sequentially, then summed them together**.**

```sql
EXPLAIN ANALYZE
SELECT COUNT(*)
FROM labs.sales_info_simple;
```

```
QUERY PLAN
Finalize Aggregate  (cost=107139.46..107139.47 rows=1 width=8) (actual time=261.738..293.345 rows=1.00 loops=1)
  Buffers: shared hit=1399 read=52656
  ->  Gather  (cost=107139.25..107139.46 rows=2 width=8) (actual time=260.311..293.340 rows=3.00 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=1399 read=52656
        ->  Partial Aggregate  (cost=106139.25..106139.26 rows=1 width=8) (actual time=237.940..237.941 rows=1.00 loops=3)
              Buffers: shared hit=1399 read=52656
              ->  Parallel Seq Scan on sales_info_simple  (cost=0.00..95722.40 rows=4166740 width=0) (actual time=0.358..121.403 rows=3333333.33 loops=3)
                    Buffers: shared hit=1399 read=52656
Planning Time: 0.060 ms
Execution Time: 293.387 ms
```

```sql
EXPLAIN ANALYZE
SELECT COUNT(*)
FROM labs.sales_info_dp;
```

```
QUERY PLAN
Finalize Aggregate  (cost=128867.92..128867.93 rows=1 width=8) (actual time=421.550..437.874 rows=1.00 loops=1)
  Buffers: shared hit=13713 read=40374
  ->  Gather  (cost=128867.71..128867.92 rows=2 width=8) (actual time=421.320..437.867 rows=3.00 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=13713 read=40374
        ->  Partial Aggregate  (cost=127867.71..127867.72 rows=1 width=8) (actual time=397.398..397.403 rows=1.00 loops=3)
              Buffers: shared hit=13713 read=40374
              ->  Parallel Append  (cost=0.00..117449.76 rows=4167179 width=0) (actual time=0.348..281.743 rows=3333333.33 loops=3)
                    Buffers: shared hit=13713 read=40374
                    ->  Parallel Seq Scan on sales_info_dp_2023_list2 sales_info_dp_8  (cost=0.00..12491.89 rows=543589 width=0) (actual time=0.533..68.213 rows=1304614.00 loops=1)
                          Buffers: shared hit=3934 read=3122
                    ->  Parallel Seq Scan on sales_info_dp_2021_list2 sales_info_dp_2  (cost=0.00..12480.65 rows=543265 width=0) (actual time=0.479..60.694 rows=1303216.00 loops=1)
                          Buffers: shared hit=94 read=6954
                    ->  Parallel Seq Scan on sales_info_dp_2022_list2 sales_info_dp_5  (cost=0.00..12474.12 rows=542812 width=0) (actual time=0.593..59.165 rows=1302750.00 loops=1)
                          Buffers: shared hit=214 read=6832
                    ->  Parallel Seq Scan on sales_info_dp_2022_list1 sales_info_dp_4  (cost=0.00..11255.88 rows=489788 width=0) (actual time=0.462..53.413 rows=1175491.00 loops=1)
                          Buffers: shared hit=581 read=5777
                    ->  Parallel Seq Scan on sales_info_dp_2023_list1 sales_info_dp_7  (cost=0.00..11233.01 rows=488801 width=0) (actual time=0.239..18.329 rows=391041.00 loops=3)
                          Buffers: shared hit=1504 read=4841
                    ->  Parallel Seq Scan on sales_info_dp_2021_list1 sales_info_dp_1  (cost=0.00..11205.45 rows=487745 width=0) (actual time=0.436..26.249 rows=584987.00 loops=2)
                          Buffers: shared hit=470 read=5858
                    ->  Parallel Seq Scan on sales_info_dp_2024_list2 sales_info_dp_11  (cost=0.00..10461.63 rows=455263 width=0) (actual time=0.078..46.700 rows=1092631.00 loops=1)
                          Buffers: shared hit=1132 read=4777
                    ->  Parallel Seq Scan on sales_info_dp_2024_list1 sales_info_dp_10  (cost=0.00..9382.60 rows=408260 width=0) (actual time=0.033..41.917 rows=979825.00 loops=1)
                          Buffers: shared hit=3132 read=2168
                    ->  Parallel Seq Scan on sales_info_dp_2021_default sales_info_dp_3  (cost=0.00..1473.08 rows=76708 width=0) (actual time=0.003..6.693 rows=130404.00 loops=1)
                          Buffers: shared hit=661 read=45
                    ->  Parallel Seq Scan on sales_info_dp_2023_default sales_info_dp_9  (cost=0.00..1466.94 rows=76394 width=0) (actual time=0.003..6.067 rows=129869.00 loops=1)
                          Buffers: shared hit=703
                    ->  Parallel Seq Scan on sales_info_dp_2022_default sales_info_dp_6  (cost=0.00..1463.41 rows=76241 width=0) (actual time=0.005..6.090 rows=129610.00 loops=1)
                          Buffers: shared hit=701
                    ->  Parallel Seq Scan on sales_info_dp_2024_default sales_info_dp_12  (cost=0.00..1225.19 rows=63819 width=0) (actual time=0.028..6.800 rows=108493.00 loops=1)
                          Buffers: shared hit=587
Planning Time: 0.139 ms
Execution Time: 437.944 ms
```

- **Count of rows with range of dates**

Flat Table: Read the entire flat table via a Parallel Sequential Scan to filter and count.

Partitioned Table: Scanned and counted rows inside only the 3 subpartitions for 2023

```sql
EXPLAIN ANALYZE
SELECT COUNT(*)
FROM labs.sales_info_simple
WHERE eventdate BETWEEN '2023-01-01' AND '2023-12-31';
```

1 | Results 2 | Results 6 | Results 4 | Results 5 | Results 6 | Results 10 | Results 8 | Results 9

N ANALYZE SELECT COUNT(*) FROM labs.sales_info_simpl | Enter a SQL expression to filter results (use Ctrl+Space)

```
QUERY PLAN
Finalize Aggregate  (cost=119935.20..119935.21 rows=1 width=8) (actual time=260.368..294.008 rows=1.00 loops=1)
  Buffers: shared hit=1598 read=52457
  ->  Gather  (cost=119934.98..119935.19 rows=2 width=8) (actual time=259.538..294.001 rows=3.00 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=1598 read=52457
        ->  Partial Aggregate  (cost=118934.98..118934.99 rows=1 width=8) (actual time=234.732..234.732 rows=1.00 loops=3)
              Buffers: shared hit=1598 read=52457
              ->  Parallel Seq Scan on sales_info_simple  (cost=0.00..116556.09 rows=951556 width=0) (actual time=122.203..209.886 rows=760182.67 loops=3)
                    Filter: ((eventdate >= '2023-01-01'::date) AND (eventdate <= '2023-12-31'::date))
                    Rows Removed by Filter: 2573151
                    Buffers: shared hit=1598 read=52457
Planning Time: 0.066 ms
Execution Time: 294.053 ms
```

```sql
EXPLAIN ANALYZE
SELECT COUNT(*)
FROM labs.sales_info_dp
WHERE eventdate BETWEEN '2023-01-01' AND '2023-12-31';
```

lts 1 | Results 2 | Results 6 | Results 4 | Results 5 | Results 6 | Results 10 | Results 8 | Results 9 | Results 10

.AIN ANALYZE SELECT COUNT(*) FROM labs.sales_info_dp W | Enter a SQL expression to filter results (use Ctrl+Space)

```
QUERY PLAN
Finalize Aggregate  (cost=39883.16..39883.17 rows=1 width=8) (actual time=172.726..203.193 rows=1.00 loops=1)
  Buffers: shared hit=6054 read=8050
  ->  Gather  (cost=39882.95..39883.16 rows=2 width=8) (actual time=172.697..203.166 rows=3.00 loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=6054 read=8050
        ->  Partial Aggregate  (cost=38882.95..38882.96 rows=1 width=8) (actual time=146.524..146.524 rows=1.00 loops=3)
              Buffers: shared hit=6054 read=8050
              ->  Parallel Append  (cost=0.00..36167.22 rows=1086292 width=0) (actual time=0.551..116.519 rows=869202.00 loops=3)
                    Buffers: shared hit=6054 read=8050
                    ->  Parallel Seq Scan on sales_info_dp_2023_list2 sales_info_dp_2  (cost=0.00..15209.84 rows=543485 width=0) (actual time=0.269..40.714 rows=434871.33 loops=3)
                          Filter: ((eventdate >= '2023-01-01'::date) AND (eventdate <= '2023-12-31'::date))
                          Buffers: shared hit=3565 read=3491
                    ->  Parallel Seq Scan on sales_info_dp_2023_list1 sales_info_dp_1  (cost=0.00..13677.02 rows=488705 width=0) (actual time=0.422..57.197 rows=586561.50 loops=2)
                          Filter: ((eventdate >= '2023-01-01'::date) AND (eventdate <= '2023-12-31'::date))
                          Buffers: shared hit=1786 read=4559
                    ->  Parallel Seq Scan on sales_info_dp_2023_default sales_info_dp_3  (cost=0.00..1848.90 rows=76379 width=0) (actual time=0.012..12.908 rows=129869.00 loops=1)
                          Filter: ((eventdate >= '2023-01-01'::date) AND (eventdate <= '2023-12-31'::date))
                          Buffers: shared hit=703
Planning Time: 0.172 ms
Execution Time: 203.242 ms
```

The partition is splitted

SELECT parentrelid::regclass AS parent_year_table, relid::regclass A | Enter a SQL expression to filter results (use Ct

	parent_year_table	category_partition_table	category_list_values
1	labs.sales_info_dp_2021	labs.sales_info_dp_2021_list2	FOR VALUES IN ('F', 'H', 'I', 'J')
2	labs.sales_info_dp_2021	labs.sales_info_dp_2021_default	DEFAULT
3	labs.sales_info_dp_2021	labs.sales_info_dp_2021_abc	FOR VALUES IN ('A', 'B', 'C')
4	labs.sales_info_dp_2021	labs.sales_info_dp_2021_de	FOR VALUES IN ('D', 'E')
5	labs.sales_info_dp_2022	labs.sales_info_dp_2022_list1	FOR VALUES IN ('A', 'B', 'C', 'D', 'E')
6	labs.sales_info_dp_2022	labs.sales_info_dp_2022_list2	FOR VALUES IN ('F', 'H', 'I', 'J')
7	labs.sales_info_dp_2022	labs.sales_info_dp_2022_default	DEFAULT
8	labs.sales_info_dp_2023	labs.sales_info_dp_2023_list1	FOR VALUES IN ('A', 'B', 'C', 'D', 'E')
9	labs.sales_info_dp_2023	labs.sales_info_dp_2023_list2	FOR VALUES IN ('F', 'H', 'I', 'J')
10	labs.sales_info_dp_2023	labs.sales_info_dp_2023_default	DEFAULT
11	labs.sales_info_dp_2024	labs.sales_info_dp_2024_list1	FOR VALUES IN ('A', 'B', 'C', 'D', 'E')
12	labs.sales_info_dp_2024	labs.sales_info_dp_2024_list2	FOR VALUES IN ('F', 'H', 'I', 'J')
13	labs.sales_info_dp_2024	labs.sales_info_dp_2024_default	DEFAULT

And reverted back

parent_year_table	category_partition_table	category_list_values
labs.sales_info_dp_2021	labs.sales_info_dp_2021_list2	FOR VALUES IN ('F', 'H', 'I', 'J')
labs.sales_info_dp_2021	labs.sales_info_dp_2021_default	DEFAULT
labs.sales_info_dp_2021	labs.sales_info_dp_2021_list1	FOR VALUES IN ('A', 'B', 'C', 'D', 'E')
labs.sales_info_dp_2022	labs.sales_info_dp_2022_list1	FOR VALUES IN ('A', 'B', 'C', 'D', 'E')
labs.sales_info_dp_2022	labs.sales_info_dp_2022_list2	FOR VALUES IN ('F', 'H', 'I', 'J')
labs.sales_info_dp_2022	labs.sales_info_dp_2022_default	DEFAULT
labs.sales_info_dp_2023	labs.sales_info_dp_2023_list1	FOR VALUES IN ('A', 'B', 'C', 'D', 'E')
labs.sales_info_dp_2023	labs.sales_info_dp_2023_list2	FOR VALUES IN ('F', 'H', 'I', 'J')
labs.sales_info_dp_2023	labs.sales_info_dp_2023_default	DEFAULT
labs.sales_info_dp_2024	labs.sales_info_dp_2024_list1	FOR VALUES IN ('A', 'B', 'C', 'D', 'E')
labs.sales_info_dp_2024	labs.sales_info_dp_2024_list2	FOR VALUES IN ('F', 'H', 'I', 'J')
labs.sales_info_dp_2024	labs.sales_info_dp_2024_default	DEFAULT

--TASK 3 USE PARALLEL QUERING

a. Select all from tables
The flat table scanned with parallel Seq Scan
Parallel Append is observed on both partitioned tables, instead of one worker scan one child at a time, workers dynamically grabbed child/partition.

```
Gather  (cost=0.00..79055.44 rows=10000175 width=11) (actual time=0.564..1185.329 rows=10000000.00 loops=1)
  Output: id, category, ischeck, eventdate
  Workers Planned: 4
  Workers Launched: 4
  Buffers: shared hit=490 read=53565
  ->  Parallel Seq Scan on labs.sales_info_simple  (cost=0.00..79055.44 rows=2500044 width=11) (actual time=0.523..74.778 rows=2000000.00 loops=5)
        Output: id, category, ischeck, eventdate
        Buffers: shared hit=490 read=53565
        Worker 0:  actual time=0.422..92.309 rows=2497685.00 loops=1
          Buffers: shared hit=113 read=13388
        Worker 1:  actual time=1.175..92.918 rows=2470305.00 loops=1
          Buffers: shared hit=134 read=13219
        Worker 2:  actual time=0.375..85.350 rows=2313610.00 loops=1
          Buffers: shared hit=125 read=12381
        Worker 3:  actual time=0.484..94.650 rows=2558560.00 loops=1
          Buffers: shared hit=118 read=13713
Planning Time: 0.053 ms
Execution Time: 1416.389 ms
```

A-Z QUERY PLAN

```
Gather  (cost=0.00..91586.33 rows=10000621 width=11) (actual time=0.690..1110.101 rows=10000000.00 loops=1)
  Output: sales_info_dp.id, sales_info_dp.category, sales_info_dp.ischeck, sales_info_dp.eventdate
  Workers Planned: 4
  Workers Launched: 4
  Buffers: shared hit=10996 read=43088
  ->  Parallel Append  (cost=0.00..91586.33 rows=2500156 width=11) (actual time=0.715..160.081 rows=2000000.00 loops=5)
        Buffers: shared hit=10996 read=43088
        Worker 0:  actual time=0.369..179.041 rows=2313097.00 loops=1
          Buffers: shared hit=1418 read=11094
        Worker 1:  actual time=1.621..196.398 rows=2515075.00 loops=1
          Buffers: shared hit=1212 read=12391
        Worker 2:  actual time=0.909..209.545 rows=2515189.00 loops=1
          Buffers: shared hit=774 read=12829
        Worker 3:  actual time=0.670..202.531 rows=2507518.00 loops=1
          Buffers: shared hit=6785 read=6774
        ->  Parallel Seq Scan on labs.sales_info_dp_2023_list2 sales_info_dp_8  (cost=0.00..10317.53 rows=326154 width=11) (actu
              Output: sales_info_dp_8.id, sales_info_dp_8.category, sales_info_dp_8.ischeck, sales_info_dp_8.eventdate
              Buffers: shared hit=282 read=6774
              Worker 3:  actual time=0.668..52.324 rows=1304614.00 loops=1
                Buffers: shared hit=282 read=6774
        ->  Parallel Seq Scan on labs.sales_info_dp_2021_list2 sales_info_dp_2  (cost=0.00..10307.59 rows=325959 width=11) (actu
              Output: sales_info_dp_2.id, sales_info_dp_2.category, sales_info_dp_2.ischeck, sales_info_dp_2.eventdate
              Buffers: shared hit=282 read=6766
              Worker 2:  actual time=0.907..54.939 rows=1303216.00 loops=1
                Buffers: shared hit=282 read=6766
        ->  Parallel Seq Scan on labs.sales_info_dp_2022_list2 sales_info_dp_5  (cost=0.00..10302.88 rows=325688 width=11) (actu
              Output: sales_info_dp_5.id, sales_info_dp_5.category, sales_info_dp_5.ischeck, sales_info_dp_5.eventdate
```

```
QUERY PLAN
Gather  (cost=0.00..73912.97 rows=8072708 width=11) (actual time=4.105..914.086 rows=8070898.00 loops=1)
  Output: sales_info.id, sales_info.category, sales_info.ischeck, sales_info.eventdate
  Workers Planned: 4
  Workers Launched: 4
  Buffers: shared hit=1881 read=41749
  ->  Parallel Append  (cost=0.00..73912.97 rows=2018178 width=11) (actual time=0.414..130.327 rows=1614179.60 loops=5)
        Buffers: shared hit=1881 read=41749
        Worker 0:  actual time=0.544..161.797 rows=2046206.00 loops=1
          Buffers: shared hit=240 read=10822
        Worker 1:  actual time=0.457..163.601 rows=2044805.00 loops=1
          Buffers: shared hit=226 read=10827
        Worker 2:  actual time=0.545..166.739 rows=2053889.00 loops=1
          Buffers: shared hit=467 read=10636
        Worker 3:  actual time=0.513..144.386 rows=1783917.00 loops=1
          Buffers: shared hit=947 read=8696
        ->  Parallel Seq Scan on public.sales_info_2024 sales_info_4  (cost=0.00..18093.64 rows=572164 width=11) (actual time=0
              Output: sales_info_4.id, sales_info_4.category, sales_info_4.ischeck, sales_info_4.eventdate
              Buffers: shared hit=470 read=11902
              Worker 2:  actual time=0.543..76.938 rows=1819494.00 loops=1
                Buffers: shared hit=236 read=9600
              Worker 3:  actual time=0.786..20.698 rows=469160.00 loops=1
                Buffers: shared hit=234 read=2302
        ->  Parallel Seq Scan on public.sales_info_2025 sales_info_3  (cost=0.00..18036.68 rows=570368 width=11) (actual time=0
              Output: sales_info_3.id, sales_info_3.category, sales_info_3.ischeck, sales_info_3.eventdate
              Buffers: shared hit=470 read=11863
              Worker 0:  actual time=0.003..0.032 rows=793.00 loops=1
                Buffers: shared hit=5
```

b. Add order by eventadate

Despite being range-partitioned by the exact column being sorted (eventdate),partitioned tables did not get a Merge Append sort and processed just like the unpartitioned table, and in fact finished slower

Example:

```
QUERY PLAN
Gather Merge  (cost=354247.91..473196.36 rows=8072708 width=11) (actual time=689.409..1965.341 rows=8070898.00 loc
  Output: sales_info.id, sales_info.category, sales_info.ischeck, sales_info.eventdate
  Workers Planned: 4
  Workers Launched: 4
  Buffers: shared hit=1187 read=42595, temp read=43487 written=43598
  ->  Sort  (cost=354247.85..359293.29 rows=2018178 width=11) (actual time=639.660..762.115 rows=1614179.60 loops=5)
        Output: sales_info.id, sales_info.category, sales_info.ischeck, sales_info.eventdate
        Sort Key: sales_info.eventdate
        Sort Method: external merge  Disk: 37264kB
        Buffers: shared hit=1187 read=42595, temp read=43487 written=43598
        Worker 0:  actual time=628.991..748.200 rows=1543138.00 loops=1
          Sort Method: external merge  Disk: 33296kB
          Buffers: shared hit=248 read=8132, temp read=8315 written=8336
        Worker 1:  actual time=622.731..741.801 rows=1577699.00 loops=1
          Sort Method: external merge  Disk: 34040kB
          Buffers: shared hit=379 read=8188, temp read=8501 written=8523
        Worker 2:  actual time=625.257..745.163 rows=1567743.00 loops=1
          Sort Method: external merge  Disk: 33824kB
          Buffers: shared hit=254 read=8259, temp read=8448 written=8469
        Worker 3:  actual time=632.580..758.846 rows=1654825.00 loops=1
          Sort Method: external merge  Disk: 35696kB
```

c. Select count of all rows

All three tables produced the expected two-phase parallel aggregation (Partial Aggregate on each of the 4 workers and single Finalize Aggregate combining the partials).Full-table COUNT(*) has nothing from partitioning

```
QUERY PLAN
Finalize Aggregate  (cost=78958.44..78958.45 rows=1 width=8) (actual time=579.802..599.975 rows=1.00 loops=1)
  Output: count(*)
  Buffers: shared hit=1611 read=42019
  ->  Gather  (cost=78958.42..78958.43 rows=4 width=8) (actual time=579.602..599.966 rows=5.00 loops=1)
        Output: (PARTIAL count(*))
        Workers Planned: 4
        Workers Launched: 4
        Buffers: shared hit=1611 read=42019
        ->  Partial Aggregate  (cost=78958.42..78958.43 rows=1 width=8) (actual time=464.034..464.036 rows=1.00 loops=5)
              Output: PARTIAL count(*)
              Buffers: shared hit=1611 read=42019
              Worker 0:  actual time=446.058..446.060 rows=1.00 loops=1
                Buffers: shared hit=267 read=8459
              Worker 1:  actual time=428.054..428.056 rows=1.00 loops=1
                Buffers: shared hit=324 read=7783
              Worker 2:  actual time=431.243..431.245 rows=1.00 loops=1
                Buffers: shared hit=406 read=7551
              Worker 3:  actual time=438.591..438.593 rows=1.00 loops=1
                Buffers: shared hit=87 read=8473
              ->  Parallel Append  (cost=0.00..73912.97 rows=2018178 width=0) (actual time=1.625..362.033 rows=1614179.60 l
                    Buffers: shared hit=1611 read=42019
                    Worker 0:  actual time=2.075..351.050 rows=1614144.00 loops=1
                      Buffers: shared hit=267 read=8459
                    Worker 1:  actual time=2.383..335.266 rows=1499757.00 loops=1
                      Buffers: shared hit=324 read=7783
                    Worker 2:  actual time=1.829..339.805 rows=1471913.00 loops=1
                      Buffers: shared hit=406 read=7551
```

d. Add range of dates

Each partitioned table only scanned the rows physically in the relevant year rather than filtering all 10M rows

Example:

QUERY PLAN

```
-> Parallel Seq Scan on labs.sales_info_dp_2024_list2 sales_info_dp_2  (cost=0.00..10006.37 rows=160985 width=11) (actual time=0.225..31.176 rows=162773.50 loops=4)
     Output: sales_info_dp_2.id, sales_info_dp_2.category, sales_info_dp_2.ischeck, sales_info_dp_2.eventdate
     Filter: ((sales_info_dp_2.eventdate >= '2024-01-01'::date) AND (sales_info_dp_2.eventdate <= '2024-06-30'::date))
     Rows Removed by Filter: 110384
     Buffers: shared hit=967 read=4942
     Worker 0:  actual time=0.431..59.967 rows=313360.00 loops=1
       Buffers: shared hit=293 read=2554
     Worker 1:  actual time=0.005..4.583 rows=19901.00 loops=1
       Buffers: shared hit=169 read=12
     Worker 2:  actual time=0.461..55.528 rows=297668.00 loops=1
       Buffers: shared hit=327 read=2370
     Worker 3:  actual time=0.005..4.627 rows=20165.00 loops=1
       Buffers: shared hit=178 read=6
-> Parallel Seq Scan on labs.sales_info_dp_2024_list1 sales_info_dp_1  (cost=0.00..8974.34 rows=145782 width=11) (actual time=0.639..36.593 rows=194472.67 loops=3)
     Output: sales_info_dp_1.id, sales_info_dp_1.category, sales_info_dp_1.ischeck, sales_info_dp_1.eventdate
     Filter: ((sales_info_dp_1.eventdate >= '2024-01-01'::date) AND (sales_info_dp_1.eventdate <= '2024-06-30'::date))
     Rows Removed by Filter: 132136
     Buffers: shared hit=564 read=4736
     Worker 1:  actual time=0.619..50.050 rows=272687.00 loops=1
       Buffers: shared hit=281 read=2198
     Worker 3:  actual time=1.139..53.800 rows=281920.00 loops=1
       Buffers: shared hit=283 read=2274
-> Parallel Seq Scan on labs.sales_info_dp_2024_default sales_info_dp_3  (cost=0.00..993.85 rows=16278 width=11) (actual time=0.021..13.295 rows=64756.00 loops=1)
     Output: sales_info_dp_3.id, sales_info_dp_3.category, sales_info_dp_3.ischeck, sales_info_dp_3.eventdate
     Filter: ((sales_info_dp_3.eventdate >= '2024-01-01'::date) AND (sales_info_dp_3.eventdate <= '2024-06-30'::date))
     Rows Removed by Filter: 43737
     Buffers: shared hit=587
```

QUERY PLAN

```
Gather  (cost=0.00..91555.66 rows=1140634 width=11) (actual time=15.454..328.373 rows=1140329.00 loops=1)
  Output: id, category, ischeck, eventdate
  Workers Planned: 4
  Workers Launched: 4
  Buffers: shared hit=1315 read=52740
  -> Parallel Seq Scan on labs.sales_info_simple  (cost=0.00..91555.66 rows=285158 width=11) (actual time=39.055..154.021 rows=228065.80 loops=5)
       Output: id, category, ischeck, eventdate
       Filter: ((sales_info_simple.eventdate >= '2024-01-01'::date) AND (sales_info_simple.eventdate <= '2024-06-30'::date))
       Rows Removed by Filter: 1771934
       Buffers: shared hit=1315 read=52740
       Worker 0:  actual time=88.662..196.026 rows=315298.00 loops=1
         Buffers: shared hit=131 read=13344
       Worker 1:  actual time=90.064..196.903 rows=311640.00 loops=1
         Buffers: shared hit=128 read=12179
       Worker 2:  actual time=1.177..91.038 rows=215303.00 loops=1
         Buffers: shared hit=67 read=6412
       Worker 3:  actual time=0.463..103.524 rows=282211.00 loops=1
         Buffers: shared hit=77 read=7434
Planning Time: 0.056 ms
Execution Time: 357.592 ms
```

e. Add grouping by category

The flat table again shows same pattern as SELECT * and COUNT(*). Since there's no WHERE category = ... filter, the partitioning by category shows no benefit.

Examples:

```
QUERY PLAN
Finalize GroupAggregate  (cost=104096.81..104114.60 rows=200 width=10) (actual time=523.489..546.481 rows=10.00 loops=1)
  Output: sales_info_dp.category, count(*)
  Group Key: sales_info_dp.category
  Buffers: shared hit=10779 read=43337
  ->  Gather Merge  (cost=104096.81..104108.60 rows=800 width=10) (actual time=523.483..546.472 rows=42.00 loops=1)
        Output: sales_info_dp.category, (PARTIAL count(*))
        Workers Planned: 4
        Workers Launched: 4
        Buffers: shared hit=10779 read=43337
        ->  Sort  (cost=104096.76..104097.26 rows=200 width=10) (actual time=470.137..470.140 rows=8.40 loops=5)
              Output: sales_info_dp.category, (PARTIAL count(*))
              Sort Key: sales_info_dp.category
              Sort Method: quicksort  Memory: 25kB
              Buffers: shared hit=10779 read=43337
              Worker 0:  actual time=482.826..482.829 rows=9.00 loops=1
                Sort Method: quicksort  Memory: 25kB
                Buffers: shared hit=1520 read=9431
              Worker 1:  actual time=485.797..485.799 rows=9.00 loops=1
                Sort Method: quicksort  Memory: 25kB
                Buffers: shared hit=1306 read=9975
              Worker 2:  actual time=476.319..476.323 rows=9.00 loops=1
                Sort Method: quicksort  Memory: 25kB
                Buffers: shared hit=1774 read=8646
              Worker 3:  actual time=382.850..382.852 rows=5.00 loops=1
```

```
QUERY PLAN
Gather  (cost=0.00..91555.66 rows=1140634 width=11) (actual time=15.454..328.373 rows=1140329.00 loops=1)
  Output: id, category, ischeck, eventdate
  Workers Planned: 4
  Workers Launched: 4
  Buffers: shared hit=1315 read=52740
  ->  Parallel Seq Scan on labs.sales_info_simple  (cost=0.00..91555.66 rows=285158 width=11) (actual time=39.055..154.021 rows=228065.80 loops=5)
        Output: id, category, ischeck, eventdate
        Filter: ((sales_info_simple.eventdate >= '2024-01-01'::date) AND (sales_info_simple.eventdate <= '2024-06-30'::date))
        Rows Removed by Filter: 1771934
        Buffers: shared hit=1315 read=52740
        Worker 0:  actual time=88.662..196.026 rows=315298.00 loops=1
          Buffers: shared hit=131 read=13344
        Worker 1:  actual time=90.064..196.903 rows=311640.00 loops=1
          Buffers: shared hit=128 read=12179
        Worker 2:  actual time=1.177..91.038 rows=215303.00 loops=1
          Buffers: shared hit=67 read=6412
        Worker 3:  actual time=0.463..103.524 rows=282211.00 loops=1
          Buffers: shared hit=77 read=7434
Planning Time: 0.056 ms
Execution Time: 357.592 ms
```

f. Join SALES_INFO and SALES_INFO_DP on id and count rows on exact date.

```sql
SELECT COUNT(*)
FROM labs.SALES_INFO s
JOIN labs.SALES_INFO_DP d ON s
WHERE s.eventdate = '2024-03-1
```

1 | Results 2 | Results 3

COUNT(*) FROM labs.SALES_INFO s JOIN

123 count
6,250

Adding indexes:

The index provided no net benefit for any of the query.On ORDER BY eventdate, index *was* used (enabled Merge Append instead of a sort), but it lost parallelism entirely.

```
QUERY PLAN
Append  (cost=4.68..580244.88 rows=10000000 width=11) (actual time=0.253..5226.543 rows=10000000.00 loops=1)
  Buffers: shared hit=7714523 read=71201 written=1
  ->  Merge Append  (cost=1.17..138044.61 rows=2603594 width=11) (actual time=0.253..1280.100 rows=2603594.00 loops=1)
        Sort Key: sales_info_dp_2.eventdate
        Buffers: shared hit=2030955 read=16045 written=1
        ->  Index Scan using sales_info_dp_2021_list1_eventdate_idx on labs.sales_info_dp_2021_list1 sales_info_dp_2  (cost=0.43..46
              Output: sales_info_dp_2.id, sales_info_dp_2.category, sales_info_dp_2.ischeck, sales_info_dp_2.eventdate
              Index Searches: 1
              Buffers: shared hit=912309 read=7399
        ->  Index Scan using sales_info_dp_2021_list2_eventdate_idx on labs.sales_info_dp_2021_list2 sales_info_dp_3  (cost=0.43..52
              Output: sales_info_dp_3.id, sales_info_dp_3.category, sales_info_dp_3.ischeck, sales_info_dp_3.eventdate
              Index Searches: 1
              Buffers: shared hit=1016307 read=8337 written=1
        ->  Index Scan using sales_info_dp_2021_default_eventdate_idx on labs.sales_info_dp_2021_default sales_info_dp_4  (cost=0
              Output: sales_info_dp_4.id, sales_info_dp_4.category, sales_info_dp_4.ischeck, sales_info_dp_4.eventdate
              Index Searches: 1
```